

packages/graph-core/src/analyzer/llm-analyzer.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T19:57:27.922Z

Overview

A new file `packages/graph-core/src/analyzer/llm-analyzer.ts` (added lines 1-186) introduces helpers for LLM-based analysis of source files and projects. It defines two result interfaces, prompt builders, a JSON extractor, and parsers that validate and normalize LLM responses.

Key changes

- **Interfaces**
 - `LLMFileAnalysis` (lines 1-9) – `fileSummary`, `tags`, `complexity`, `functionSummaries`, `classSummaries`, optional `languageNotes`.
 - `LLMProjectSummary` (lines 10-18) – `description`, `frameworks`, `layers` (each with `name`, `description`, `filePatterns`).
- **Prompt builders**
 - `buildFileAnalysisPrompt` (lines 19-42) – constructs a prompt that asks the LLM to return a JSON object with file-level metadata.
 - `buildProjectSummaryPrompt` (lines 48-75) – builds a prompt for a project-wide summary, including a file list and optional sample snippets.
- **JSON extraction** – `extractJson` (lines 81-95) removes markdown fences or extracts raw JSON from an LLM reply.
- **Response parsers**
 - `parseFileAnalysisResponse` (lines 102-142) – parses the file-analysis JSON, normalizes `complexity` against `VALID_COMPLEXITIES`, and returns `null` on failure.
 - `parseProjectSummaryResponse` (lines 148-185) – parses the project-summary JSON, filters arrays for string safety, and maps layer objects; returns `null` on failure.

Impact

- **Type safety & runtime checks** – The interfaces and parsing logic enforce expected shapes, reducing the risk of downstream errors from malformed LLM output.
- **Centralized prompt logic** – All prompt wording and response handling live in one module, simplifying future updates.
- **Graceful failure** – Parsers return `null` when JSON is invalid, allowing callers to log or retry without crashing.

Risks & follow-ups

- **LLM response drift** – If the LLM changes its JSON structure, parsers may return `null`. Verify against current LLM outputs.
- **Complexity validation** – `VALID_COMPLEXITIES` is hard-coded; adding new levels requires updating the set and prompt instructions.

- **Performance** – Large file contents in prompts increase token usage; monitor LLM cost.
- **Testing** – Add unit tests covering missing fields and unexpected types to catch regressions early.